Query Reliability Mechanism

Table of Contents

- I. Overall Architecture Overview
  - 1.1 Design Philosophy Overview: Query as a State Machine
  - 1.2 Dual-Layer Entry Functions
  - 1.3 Design Intent of Loop State (State)
- II. Context Compression & Overflow Recovery
  - 2.1 Proactive Compression Chain (Proactive → Autocompact)
  - 2.2 Reactive Compression Chain (Reactive)
  - 2.3 Blocking Limit Check
- III. API Retry & Model Fault Tolerance
  - 3.0 Where the Two-Layer Error Handling Boundary Lies
  - 3.1 `withRetry` Wrapper (`services/api/withRetry.ts`)
  - 3.2 Model Fallback
- IV. Streaming Fault Tolerance
  - 4.1 Streaming Tool Executor
  - 4.2 Orphan Message Handling (Tombstoning)
  - 4.3 Tool Result Budget (Tool Result Size Limit)
- V. Max Output Tokens Recovery
  - 5.1 Level 1: Token Cap Upgrade
  - 5.2 Level 2: Multi-Turn Recovery (`MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3`)
  - 5.3 Level 3: Expose Error
- VI. Token Budget Mechanism
  - 6.1 Design Intent of Diminishing Returns Detection
  - 6.2 `lastGlobalTurnTokens` & Inter-Turn Accumulation Tracking
- VII. Stop Hooks Mechanism
  - 7.1 Execution Flow
  - 7.2 Blocking Handling
  - 7.3 Prevent Continuation
  - 7.4 Teammate Hook Chain
- VIII. User Interruption Handling
  - 8.1 Streaming Interruption (User Ctrl+C)
  - 8.2 Tool Execution Interruption
  - 8.3 Submit Interrupt
- IX. Async Prefetch & Parallelization
  - 9.1 Memory Prefetch
  - 9.2 Skill Discovery Prefetch
  - 9.3 Tool Use Summary
- X. Tool Use Context Refresh
- XI. Error Classification & Handling

- 11.1 Error Type Mapping
- 11.2 Safety Guarantees for Error Exposure
- XII. Feature Flag Summary
  - 12.1 `querySource` as a Typed Safety Policy Context
- XIII. Design Philosophy Summary
  - 13.0 The "Three-Layer Funnel" Philosophy of Error Handling
  - 13.1 Defensive Streaming Processing
  - 13.2 Withheld Mode
  - 13.3 Immutable State Transitions
  - 13.4 Multi-Layer Degradation Strategy
  - 13.5 Dependency Injection: Intentional Design for Testability
  - 13.6 Protection Against 529 Overload "Amplification Attacks"
  - 13.7 Avalanche Isolation for Background Tasks
  - 13.8 QueryCheckpoint's Dual-Layer Performance Tracking System
  - 13.9 "Implicit Dependency" Awareness in Tool Execution
  - 13.10 Analytics as a First-Class Citizen of Architecture
  - 13.11 `using` Declaration & Deterministic Resource Management
  - 13.12 Death Spiral Protection
- XIV. Key Data Flow Diagram
- XV. Quick Reference: Related Files

I. Overall Architecture Overview

`query.ts` is Claude Code's **main query engine**, responsible for driving the complete dialogue loop between the user and the LLM. It is implemented as an `async generator` supporting streaming output, and injects extensive reliability mechanisms between each API call to handle runtime failures such as context overflow, rate limits, model overload, and user interruptions.

1.1 Design Philosophy Overview: Query as a State Machine

The key to understanding query.ts is not memorizing every recovery path, but understanding **why** it is designed as an infinite-loop **state machine** rather than a sequential function chain. Three core principles underpin this choice:

Principle 1: LLM dialogue is inherently non-deterministic. A single API call may fail due to context overrun (requiring compression), wait due to rate limits (requiring retry), need degradation due to model overload (requiring model switch), or be interrupted by the user pressing Ctrl+C (requiring cleanup). These failures cannot all be predicted before the call and must be handled one by one at runtime. Wrapping the entire flow in a `while(true)` loop gives every failure type a unified "continue iteration and retry" exit, eliminating the need for separate retry branches for each error type.

Principle 2: Streaming output demands real-time intervention. Claude Code is not a batch processing system that "waits for the model to finish a full response before processing." It **begins executing tools while the streaming output is still in progress**. This creates a fundamental tension: once you start yielding streaming data, the control flow has already diverged to the caller (SDK/REPL). The `async generator` design solves this — `yield` hands control back to the query loop while streaming output continues, allowing the loop to process

the next tool result while the caller continues consuming the already-yielded data. This is "cooperative concurrency": the generator and caller each advance, coordinating through `yield`.

Principle 3: Immutable state transitions prevent "ghost bugs". There are 9 `continue` sites in the query loop, each requiring **all** cross-iteration state to be updated. If mutable field assignments were used (e.g., `state.field = newValue`), a single `continue` site forgetting to reset a field would cause state leakage, leading to hard-to-trace ghost bugs. Replacing the **entire `State` object with a new one** means each `continue` site either explicitly sets every field (via spread) or inherits the old value (via `...(previous)`) — a missing field equals missing a line of code, making it immediately visible in code review.

Core file structure:

```
query.ts                          # Main query loop (entry + core while(true) loop)
├── query/stopHooks.ts            # Stop Hooks executor (hook chain at end of turn)
├── query/tokenBudget.ts          # Token budget tracking & early-stop judgment
├── services/api/withRetry.ts     # API call retry wrapper
└── [other supporting modules]
    autoCompact / reactiveCompact / contextCollapse / microcompact / snipCompact
    StreamingToolExecutor / runTools
```

1.2 Dual-Layer Entry Functions

query.ts deliberately separates the public API (`query`) and the internal loop body (`queryLoop`) into two functions:

```
// Public entry: responsible for cleaning up consumedCommandUuids
export async function* query(params: QueryParams): AsyncGenerator<...> {
  const consumedCommandUuids: string[] = []
  const terminal = yield* queryLoop(params, consumedCommandUuids)
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed')
  }
  return terminal
}

// Internal loop body: core carrier of all runtime reliability logic
async function* queryLoop(params, consumedCommandUuids): AsyncGenerator<...> {
  // while(true) infinite loop, continues via continue jump
  // Exits via return { reason: 'xxx' }
}
```

The `query()` layer's responsibility is "session lifecycle management": when `queryLoop` returns normally (not throw or return), it notifies all consumed commands as completed. It is the "cleanup hook" for the entire generator chain. `queryLoop()` carries all runtime reliability logic — this is the core reason for the separation. Merging the two would entangle reliability logic with session cleanup logic, increasing test and modification complexity.

On the use of `yield*`: Using `yield*` (delegate generation) rather than plain `yield` is deliberate — when an internal generator of `queryLoop` `throw`s, the exception propagates directly through `query()` to the caller, without an explicit try/catch. This provides the same exception-transparent propagation semantics as ordinary functions, allowing SDK consumers (cowork/desktop) to catch all exceptions with a unified `try/catch`.

PS:

- If an exception is thrown inside queryLoop, it appears directly in the caller's catch — the caller doesn't even know the `query` function existed
- This follows the same logic as return passing a value: `yield*` makes exceptions take the "direct穿透" path too, rather than being forwarded through a middleman

1.3 Design Intent of Loop State (State)

The entire reliability mechanism is built on **cross-iteration state persistence**. The `State` type aggregates all mutable state that needs to survive across loop iterations:

```
type State = {
  messages: Message[]                    // Conversation history
  toolUseContext: ToolUseContext         // Tool context
  autoCompactTracking: AutoCompactTrackingState | undefined  // Auto-compaction state
  maxOutputTokensRecoveryCount: number  // max_output_tokens recovery count
  hasAttemptedReactiveCompact: boolean  // Whether reactive compaction has been attempted
  maxOutputTokensOverride: number | undefined  // Output token cap override
  pendingToolUseSummary: Promise<...> | undefined  // Tool use summary (async)
  stopHookActive: boolean | undefined   // Whether a Stop Hook is active
  turnCount: number                     // Current turn count
  transition: Continue | undefined      // Reason for previous iteration's continuation
(debug/test)
}
```

Each `continue` replaces the entire state object via `state = { ...nextState }` — this is an **immutable state transition** pattern, avoiding scattered local variable assignments.

II. Context Compression & Overflow Recovery

PS: See the "Context Management" document for details

This is the most complex reliability subsystem in query.ts, divided into **proactive compression (Proactive)** and **reactive compression (Reactive)** chains.

2.1 Proactive Compression Chain (Proactive → Autocompact)

Before the API call, compression steps are executed in order:

2.1.1 History Snip (`HISTORY_SNIP` Feature Flag)

```
// restored-src/src/services/compact/snipCompact.ts
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages
  snipTokensFreed = snipResult.tokensFreed  // Passed to autocompact's threshold check
}
```

- Goal: Remove "old enough" and "no longer referenced" historical messages from the context window
- Key point: The number of tokens freed (`snipTokensFreed`) is passed to autocompact so that autocompact's threshold check reflects what snip has already removed

PS: This is a truncation feature — a rather aggressive context operation. In the current version of CC, **this feature is completely silently disabled**.

2.1.2 Microcompact

```
const microcompactResult = await deps.microcompact(messagesForQuery, toolUseContext,
querySource)
messagesForQuery = microcompactResult.messages
```

- **Cached Microcompact** (`CACHED_MICROCOMPACT` feature): Cache optimization for edit tool results, using the `cache_deleted_input_tokens` field from the API response to precisely calculate deletion amounts
- Deferred to after the API response to yield boundary messages, so the API's actual `cache_deleted_input_tokens` can be used

PS: This is the cache_edit capability. See the "Context Management" document for details

2.1.3 Context Collapse (`CONTEXT_COLLAPSE` Feature)

```
if (feature('CONTEXT_COLLAPSE') && contextCollapse) {
  const collapseResult = await contextCollapse.applyCollapsesIfNeeded(...)
  messagesForQuery = collapseResult.messages
}
```

- Collapse mechanism based on "commit logs": collapse results are persisted in a collapse store rather than directly modifying the REPL array
- This makes collapses persistent across multiple turns: each `projectView()` replays the commit log on a new entry point

2.1.4 Autocompact (Auto Compression)

```
const { compactionResult, consecutiveFailures } = await deps.autocompact(...)
```

- When token count exceeds a threshold, summarization compression is applied to early messages
- The `consecutiveFailures` field tracks consecutive failure count, used for **circuit breaker** logic
- After successful compression, the tracking state is reset:

```
tracking = {
  compacted: true,
  turnId: deps.uuid(),
  turnCounter: 0,         // Turn counter reset
  consecutiveFailures: 0, // Consecutive failure count reset
}
```

Analytics instrumentation: After successful compression, detailed token statistics are recorded (pre/post compression token counts, cache hit/miss, etc.) for cost analysis.

task_budget cross-compression boundary tracking: Serves as an upper bound guard for resource consumption, preventing the Agent from bypassing derived task limits either intentionally or unintentionally after context compression causes memory loss, ensuring that the total number of sub-agents across the entire session lifecycle (spanning any number of compressions) never exceeds the configured safety threshold.

2.2 Reactive Compression Chain (Reactive)

When proactive compression fails to prevent a **413 Prompt Too Long** error, the reactive path is triggered:

2.2.1 Withheld Mechanism

In the streaming loop, recoverable errors are **withheld**, not immediately exposed to the caller:

```
let withheld = false
if (contextCollapse?.isWithheldPromptTooLong(message, ...)) withheld = true
if (reactiveCompact?.isWithheldPromptTooLong(message)) withheld = true
if (mediaRecoveryEnabled && reactiveCompact?.isWithheldMediaSizeError(message)) withheld =
true
if (isWithheldMaxOutputTokens(message)) withheld = true
if (!withheld) { yield yieldMessage }  // Only yield when not withheld
```

> **Design Intent**: If errors are exposed too early, SDK consumers (cowork/desktop) would terminate the session when they see the `error` field, leaving recovery logic unclaimed.

2.2.2 Two-Phase Recovery

Phase 1: Context Collapse Queue Drain (triggered only when the previous turn was not `collapse_drain_retry`)

```
if (feature('CONTEXT_COLLAPSE') && state.transition?.reason !== 'collapse_drain_retry') {
  const drained = contextCollapse.recoverFromOverflow(messagesForQuery, querySource)
  if (drained.committed > 0) {
    state = { ...state, transition: { reason: 'collapse_drain_retry', committed:
drained.committed } }
    continue  // Retry API call
  }
}
```

PS: The core idea is not to compress the entire conversation, but to **replace message blocks marked as "collapsible" (e.g., intermediate results already referenced by the model and no longer needed in full) with shorter placeholders**.

Phase 2: Reactive Compact (if Phase 1 fails to resolve)

```
const compacted = await reactiveCompact.tryReactiveCompact({
  hasAttempted: hasAttemptedReactiveCompact,
  querySource,
  messages: messagesForQuery,
  ...
})
if (compacted) {
  state = { ...state, transition: { reason: 'reactive_compact_retry' } }
  continue
}
```

PS: The core idea is that if context collapse didn't solve the problem (no collapsible units, or still over limit after collapse), perform a full summarization: compress the entire prior conversation history into a short summary.

Deadlock Protection: If both recovery attempts fail, **do not** fall through to stop hooks — because the model never produced a valid response, stop hooks would create a death spiral: error → hook blocking → retry → error

→ ...

2.3 Blocking Limit Check

When **proactive compression is disabled**, a hard blocking limit check is performed before the API call:

```
if (!compactionResult && querySource !== 'compact' && querySource !== 'session_memory'
    && !(reactiveCompact?.isReactiveCompactEnabled() && isAutoCompactEnabled())
    && !collapseOwnsIt) {
  const { isAtBlockingLimit } = calculateTokenWarningState(
    tokenCountWithEstimation(messagesForQuery) - snipTokensFreed,
    toolUseContext.options.mainLoopModel,
  )
  if (isAtBlockingLimit) {
    yield createAssistantAPIErrorMessage({ content: PROMPT_TOO_LONG_ERROR_MESSAGE, error:
'invalid_request' })
    return { reason: 'blocking_limit' }
  }
}
```

- Skip conditions: compression just completed (already verified below threshold), is a compact/session_memory query, reactive compact or context collapse is enabled

---

III. API Retry & Model Fault Tolerance

3.0 Where the Two-Layer Error Handling Boundary Lies

There are actually two distinct retry layers in query.ts, **the boundary needs to be carefully understood**:

Layer 1 (`withRetry.ts`): Handles **transport layer and gateway layer** errors — 429 rate limits, 529 overload, 401/403 authentication, 408 timeout, ECONNRESET/EPIPE connection issues. The common characteristic of these errors: they are **completely unrelated to the content of the request itself**; they are purely infrastructure-level issues. Retrying the same content (identical messages) usually succeeds.

Layer 2 (query.ts loop): Handles **semantic layer** errors — 413 PTL, max_output_tokens, model degradation. The common characteristic of these errors: they are **directly related to the content of the request**, and can only be resolved by modifying the request content (compressing context, switching models, injecting recovery messages).

The rationale for this layering: **semantic layer modifications must abandon the current request (cannot simply retry)**, whereas transport layer errors **can retry the same request**.

3.1 `withRetry` Wrapper (`services/api/withRetry.ts`)

```
async function* withRetry<T>(
  getClient: () => Promise<Anthropic>,
  operation: (client, attempt, context) => Promise<T>,
  options: RetryOptions,
): AsyncGenerator<SystemAPIErrorMessage, T>
```

3.1.1 Retry Strategies

Error Type	Handling Strategy
408 Request Timeout	Retry (exponential backoff + jitter)
409 Lock Timeout	Retry
429 Rate Limit	Retry (non-subscribers only; enterprise users can retry due to PAYG)
529 Overloaded	Retry (`MAX_529_RETRIES=3` , triggers Model Fallback)
5xx Server Error	Retry
APIConnectionError (ECONNRESET/EPIPE)	Retry and disable keep-alive
401/403 Authentication Error	Clear cache + refresh token, then retry
max_tokens exceeds context limit	Dynamically lower `max_tokens` and retry

3.1.2 Backoff Formula

PS: Exponential Backoff with Jitter

```
const baseDelay = Math.min(BASE_DELAY_MS * 2^(attempt-1), maxDelayMs)  // maxDelayMs=32000
const jitter = Math.random() * 0.25 * baseDelay
return baseDelay + jitter
```

Why jitter is 0.25 rather than 0.5 (Full Jitter)? This means **average wait time is longer than the theoretical value**, not shorter. This may be an intentional design: API Gateway rate limiting is typically based on a sliding window, and a slightly longer average wait reduces the probability of triggering rate limiting again within the window. Full jitter performs better under low concurrency, but in this scenario (clients already generating significant concurrent requests), a slightly more conservative backoff strategy may be safer.

3.1.2.1 Priority of `Retry-After` Header

Notably, the backoff strategy **prioritizes the server-returned** `Retry-After` header if present:

```
if (retryAfterHeader) {
  return parseInt(retryAfterHeader, 10) * 1000  // Use server's suggestion directly
}
```

PS: The server knows its own current load situation better than the client.

3.1.3 Persistent Retry Mode (`UNATTENDED_RETRY`)

Suitable for long-running unattended sessions (e.g., background agent / CI pipeline):

```
const PERSISTENT_MAX_BACKOFF_MS = 5 * 60 * 1000  // 5-minute cap
const PERSISTENT_RESET_CAP_MS = 6 * 60 * 60 * 1000  // 6-hour cap
const HEARTBEAT_INTERVAL_MS = 30_000  // Yield a keep-alive signal every 30s
```

- Long sleeps are broken into 30-second chunks, with `AbortSignal` checked after each chunk
- During this time, `{type:'system', subtype:'api_retry'}` is periodically yielded to prevent the host from marking the session as idle

PS: The characteristic of such tasks is: **they'd rather wait for the API to recover than expect failure to occur**.

3.1.4 Fast Mode Degradation

Handling of 429/529 in Fast Mode:

```
if (retryAfterMs < SHORT_RETRY_THRESHOLD_MS) {  // < 20s
  await sleep(retryAfterMs); continue  // Keep fast mode, retry (short wait is worth it;
degradation would invalidate cache, a significant loss)
} else {
  triggerFastModeCooldown(Date.now() + cooldownMs)  // Degrade to standard speed
  retryContext.fastMode = false; continue
}
```

PS:

- Fast Mode is a reasoning acceleration feature of Claude Code; its core principle is using the API's **Prompt Caching** to reduce token consumption per request, thereby lowering latency and cost.
- 429 = you're rate-limited, wait a bit; 529 = the server is about to collapse, don't add to the chaos.

3.1.5 Foreground/Background Distinction

Only the following `querySource` values retry on 529 (user is waiting in the foreground):

```
const FOREGROUND_529_RETRY_SOURCES = new Set([
  'repl_main_thread', 'sdk', 'agent:*', 'compact', 'hook_agent',
  'hook_prompt', 'verification_agent', 'side_question', 'auto_mode', ...
])
```

Other background sources (summarization generation, title generation, etc.) **fail immediately** on 529, avoiding amplification of gateway pressure during capacity overload.

PS:

Foreground sources (user-aware, retriable) — only these 4 types:

1. REPL main loop — user is typing in the terminal, waiting to see output
2. Agent execution — includes custom agents, built-in agents, agents invoked in hooks
3. Compact — context compression, imperceptible to user but the task itself completes synchronously
4. Synchronous safety judgments — auto_mode classifier, side_question prompts, verification_agent
   Essentially: there is a real person sitting in front of a screen waiting for this result.

Background sources (user-unaware, non-retriable):

- Title/name generation: session titles, rename new names, Teleport titles
- Memory system: memory extraction, relevance search, cross-session search, session_memory
- Agent lifecycle: agent creation, agent summarization, agent recovery
- Hooks: prompt_suggestion, speculation, skill_improvement
- External integrations: Chrome MCP, web search, web_fetch, mcp_datetime_parse
- Auxiliary analysis: insights, auto_dream, away_summary, magic_docs, feedback
- Tool-related: tool_use_summary_generation, bash_extract_prefix, permission_explainer, model_validation
  Essentially: whether this request succeeds or fails, the user doesn't know and doesn't care. If it fails, swallow it silently — no need to add chaos when the API is overloaded.

3.2 Model Fallback

After 3 consecutive 529 errors, model fallback is triggered:

```
// in withRetry.ts
if (consecutive529Errors >= MAX_529_RETRIES) {
  if (options.fallbackModel) {
    throw new FallbackTriggeredError(options.model, options.fallbackModel)
  }
}

// Caught in query.ts
if (innerError instanceof FallbackTriggeredError && fallbackModel) {
  currentModel = fallbackModel
  attemptWithFallback = true  // Retry loop flag

  // Clear orphan messages
  for (const msg of assistantMessages) {
    yield { type: 'tombstone', message: msg }  // Remove from UI/Transcript
  }

  // Discard pending tool results with old tool_use_id
  streamingToolExecutor.discard()
  streamingToolExecutor = new StreamingToolExecutor(...)

  // Thinking signatures are model-bound; downgraded model doesn't have capybara signature
permissions
  if (process.env.USER_TYPE === 'ant') {
    messagesForQuery = stripSignatureBlocks(messagesForQuery)
  }

  yield createSystemMessage(`Switched to ${renderModelName(innerError.fallbackModel)} ...`,
'warning')
  continue
}
```

IV. Streaming Fault Tolerance

4.1 Streaming Tool Executor

When the `streamingToolExecution` feature is enabled, tools can be **executed in parallel** while the model is still streaming output, rather than waiting for the complete response:

```
const streamingToolExecutor = new StreamingToolExecutor(
  toolUseContext.options.tools,
  canUseTool,
  toolUseContext,
)

// Register tools in real-time in the streaming loop
for (const toolBlock of msgToolUseBlocks) {
  streamingToolExecutor.addTool(toolBlock, message)
}

// Fetch completed tool results in real-time
for (const result of streamingToolExecutor.getCompletedResults()) {
```

```
  yield result.message
  toolResults.push(result.message)
}
```

PS: When the model streams multiple parallel tool calls, **as soon as one tool_use block is fully output, that tool is executed immediately**, rather than waiting for all blocks to finish outputting before executing them all together.

4.2 Orphan Message Handling (Tombstoning)

When a streaming fallback occurs, some assistant messages (e.g., thinking blocks) have invalid signatures, and retrying directly would cause API errors:

```
if (streamingFallbackOccured) {
  for (const msg of assistantMessages) {
    yield { type: 'tombstone', message: msg }  // Remove from UI
  }
  logEvent('tengu_orphaned_messages_tombstoned', { orphanedMessageCount:
assistantMessages.length, ... })
  assistantMessages.length = 0
  streamingToolExecutor.discard()  // Discard results with old tool_use_id
  streamingToolExecutor = new StreamingToolExecutor(...)
}
```

PS:

- Streaming fallback: when an error occurs mid-stream, discard all received half-finished fragments, resend the request in non-streaming mode, and present only one complete AssistantMessage to the user.
- Because of 4.1 Streaming Tool Executor above, old tool_use_id results need to be discarded to prevent timing and execution chaos.

4.3 Tool Result Budget (Tool Result Size Limit)

```
messagesForQuery = await applyToolResultBudget(
  messagesForQuery,
  toolUseContext.contentReplacementState,
  persistReplacements ? records => recordContentReplacement(records, toolUseContext.agentId)
: undefined,
  new Set(toolUseContext.options.tools.filter(t =>
!Number.isFinite(t.maxResultSizeChars)).map(t => t.name)),
)
```

- Enforces **per-message budget** on aggregated tool result sizes
- Orthogonal to microcompact (they don't interfere because microcompact operates only by `tool_use_id`)
- Replacement records are persisted only for `agent:*` and `repl_main_thread` query sources (supporting `/resume` and AgentTool recovery)

Orthogonality of `applyToolResultBudget` and microcompact: These two compression mechanisms operate on different dimensions of messages — `applyToolResultBudget` compresses tool result **size within a single message** (by character count cap), while microcompact compresses **relationships between messages** (by tool result count threshold and cache state).

Reason for persisting only for specific querySources: Tool result replacement records need to be written to the filesystem (`recordContentReplacement` writes to sidechain or session file). For temporary subagent invocations (like `agent_summary`), these records quickly become invalid (session state is completely different after subagent execution completes), so writing to disk would be pure waste. By limiting persistence to "main sessions" and "agents with `/resume` capability", the system avoids unnecessary disk I/O.

V. Max Output Tokens Recovery

The **three-level recovery** when the model output hits the `max_output_tokens` limit:

5.1 Level 1: Token Cap Upgrade

Only when the Statsig feature `tengu_otk_slot_v1` is enabled, if the default 8k cap is detected in use, it escalates to `ESCALATED_MAX_TOKENS` (64k) and retries (within the same turn):

```
if (capEnabled && maxOutputTokensOverride === undefined
    && !process.env.CLAUDE_CODE_MAX_OUTPUT_TOKENS) {
  state = {
    ...state,
    maxOutputTokensOverride: ESCALATED_MAX_TOKENS,
    transition: { reason: 'max_output_tokens_escalate' },
  }
  continue
}
```

5.2 Level 2: Multi-Turn Recovery (`MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3`)

```
const recoveryMessage = createUserMessage({
  content: `Output token limit hit. Resume directly — no apology, no recap. ` +
           `Pick up mid-thought if that is where the cut happened. ` +
           `Break remaining work into smaller pieces.`,
  isMeta: true,
})
const next: State = {
  messages: [...messagesForQuery, ...assistantMessages, recoveryMessage],
  maxOutputTokensRecoveryCount: maxOutputTokensRecoveryCount + 1,
  ...
}
state = next; continue
```

PS: This actually uses **contextual dynamic injection/intervention**.

- **Constructs an instruction prompt**, injects it into conversation history, then continues and reruns the API: `You were cut off. Don't apologize, don't recap. Continue directly from where you were interrupted. If you're not done, break the remaining work into smaller pieces.`
- This message is for the model, not the user (isMeta: true means it doesn't appear in the UI).

5.3 Level 3: Expose Error

If all three levels are exhausted, the withheld error message is exposed to the user:

```
if (isWithheldMaxOutputTokens(lastMessage)) {
  yield lastMessage  // Expose error
}
```

VI. Token Budget Mechanism

`query/tokenBudget.ts` implements an "early stop" mechanism to prevent continued consumption when the token budget is about to be exhausted:

```
type BudgetTracker = {
  continuationCount: number   // Number of auto-continued turns
  lastDeltaTokens: number     // Token increment since last turn
  lastGlobalTurnTokens: number // Total tokens in last turn
  startedAt: number
}

function checkTokenBudget(tracker, agentId, budget, globalTurnTokens): TokenBudgetDecision
```

Continuation conditions:

```
  /**
   * Diminishing Returns Detection
   * All three conditions must be met simultaneously:
   *   1. At least 3 consecutive checks have occurred (multiple turns have already run this
round)
   *   2. Tokens produced this turn < 500 (minimal output this round)
   *   3. Tokens produced last turn < 500 (minimal output last round)
   *
   * Implication: The model has produced only small amounts of content for multiple
consecutive turns,
   * indicating the task is near completion.
   * Continuing to let the model run would only waste budget — stop this round.
   */
  const isDiminishing =
    tracker.continuationCount >= 3 &&      // ① Continuation count threshold met
    deltaSinceLastCheck < DIMINISHING_THRESHOLD &&  // ② This turn's increment < 500 tokens
    tracker.lastDeltaTokens < DIMINISHING_THRESHOLD  // ③ Last turn's increment < 500 tokens

  /**
   * Passed the check: not diminishing AND not at 90% of budget
   * → Allow the model to continue working, give the user a nudge about current consumption.
   *
   * Stop if either condition is met:
   *   - Already diminishing (model output getting shorter, near completion)
   *   - Already exceeded 90% budget (not much remaining space)
   */
  if (!isDiminishing && turnTokens < budget * COMPLETION_THRESHOLD) {  // < 90%
    return { action: 'continue', nudgeMessage: '...' }
  }
```

Stop conditions:

- Token usage exceeds 90% of budget, or
- 3 consecutive turns with increment < 500 tokens (**diminishing returns detection**), or
- No budget (agentId subagent scenario)

Meta message during auto-continuation: Shows progress percentage to the user:

```
Token budget continuation #2: 67% (335,000 / 500,000)
```

6.1 Design Intent of Diminishing Returns Detection

The "diminishing returns detection" in `checkTokenBudget` (`continuationCount >= 3 && deltaSinceLastCheck < 500`) is an **adaptive stop mechanism** that acknowledges a reality: an Agent's work output does not grow linearly with token consumption.

Specifically: the Agent solves most problems in the first few chunks ("easy wins"), and subsequent chunks mainly handle "edge cases" and "repeated verification". If the token increment is less than 500 for 3 consecutive turns, it means the model added very little new content each turn — which could mean:

1. Work is nearly complete (best case), or
2. The model is stuck in repetitive work or spinning its wheels (worst case)

The code chooses **conservative stopping** — better to terminate early than let the model consume unlimited budget. This is a pragmatic design philosophy: in the face of unpredictable Agent behavior, budget guards are more reliable than precise completion judgment.

PS:

- The more interesting design aspect is using **token output rate to determine whether the model is in a working state or an idle state**.
- But this design is somewhat brute-force -> consider using contextual dynamic injection to intervene in LLM behavior

6.2 `lastGlobalTurnTokens` & Inter-Turn Accumulation Tracking

The design of `BudgetTracker.lastGlobalTurnTokens` reveals a question: **where is token budget consumption calculated?**

`getTurnOutputTokens()` comes from `bootstrap/state.ts` and records the **total token consumption of the entire turn** (including all API rounds). But `deltaSinceLastCheck = globalTurnTokens - tracker.lastGlobalTurnTokens` calculates the **increment between two adjacent checkpoints**. This means:

- If a turn includes multiple API calls (multiple rounds of tool execution), all those calls' tokens accumulate into `globalTurnTokens`
- But `lastDeltaTokens` records only "increment from the last budget checkpoint to now", not "total consumption of the last turn"

PS: This is primarily **instantaneous change detection** — `deltaSinceLastCheck` cares about "how much more was consumed this turn compared to the last", not "what is the total for this turn". The latter is handled by the `budget * COMPLETION_THRESHOLD` absolute threshold.

VII. Stop Hooks Mechanism

`query/stopHooks.ts` executes after each tool call completes (`needsFollowUp=false`):

```
const stopHookResult = yield* handleStopHooks(
  messagesForQuery, assistantMessages, systemPrompt, userContext,
  systemContext, toolUseContext, querySource, stopHookActive,
)
```

stopHookResult structure:

```
  type StopHookResult = {
    blockingErrors: Message[]     // Error messages reported by hooks (can accumulate
multiple)
    preventContinuation: boolean // Hook explicitly requests termination
  }
```

7.1 Execution Flow

1. **Save CacheSafeParams**: Used by `/btw` and SDK `control_request`
2. **Template Job Classification**: Update `state.json` after each turn
3. **Background tasks** (fire-and-forget):
   - Prompt Suggestion
   - Memory Extraction (`EXTRACT_MEMORIES`)
   - Auto Dream
4. **Stop Hook Execution**: Core blocking logic

PS: `/btw` and SDK `control_request` need to initiate a new API request while reusing the main loop's cache — Stop Hooks secretly snapshot the main loop's API request parameters at the end of each turn, making them directly available for these two scenarios, saving the overhead of recreating the prompt cache.

- `/btw` : While the main loop is working, the user doesn't interrupt it but directly pops up a side question to get an answer
- `control_request` : SDK host applications (IDE plugins, external schedulers) can remotely send a control_request to a running Claude Code session, with the exact same effect as `/btw` : read the main loop's context, initiate a single-turn fork query, return the answer

7.2 Blocking Handling

```
if (stopHookResult.blockingErrors.length > 0) {
  state = {
    messages: [...messagesForQuery, ...assistantMessages, ...stopHookResult.blockingErrors],
    stopHookActive: true,  // Prevent infinite recursion
    transition: { reason: 'stop_hook_blocking' },
  }
  continue  // Inject blocking error into next turn; model self-corrects upon seeing it
}
```

Infinite Loop Protection: `hasAttemptedReactiveCompact` remains un-reset. If reactive compact has already failed once (indicating the context truly exceeds the limit), stop hook's blocking error triggering compact again would enter a dead loop — this protection keeps `hasAttemptedReactiveCompact` as `true` , avoiding repeated compression attempts.

Deep meaning of `hasAttemptedReactiveCompact` persistence: The semantic of this field is not "has attempted before", but "compression is ineffective for the current context". Once set to `true`, subsequent stop hook blocking errors cannot trigger new compression attempts — because it has been proven that compression cannot solve the problem, and continuing to try would only waste API calls and potentially expand the context.

This effectively creates an associative memory between stop hook blocking errors and reactive compact failures: stop hooks believe the error can be resolved by correcting the model's behavior, but if reactive compact has already shown that the problem is not about model behavior but about context scale, stop hook intervention is ineffective.

PS: REACTIVE_COMPACT is disabled by default in the current version — this is still a feature in the testing phase.

7.3 Prevent Continuation

If a hook explicitly requests stopping:

```
if (stopHookResult.preventContinuation) {
  return { reason: 'stop_hook_prevented' }
}
```

7.4 Teammate Hook Chain

For teammates in team mode, **after all regular Stop Hooks have passed** (no blocking errors, no preventContinuation), the following execute additionally:

- `TaskCompleted` hooks (for in_progress tasks owned by this teammate)
  - Query the task list, find **this teammate's in-progress tasks**
  - Execute a TaskCompleted hook for each task
- `TeammateIdle` hooks
  - After all TaskCompleted hooks finish, execute a TeammateIdle hook once. This hook is not associated with any specific task; it concerns whether this teammate as a whole is idle

---

VIII. User Interruption Handling

8.1 Streaming Interruption (User Ctrl+C)

Trigger condition: `toolUseContext.abortController.signal.aborted === true` (user pressed ESC to reject tool / Ctrl+C / SDK initiated abort)
Handling principle: Give started tools a proper conclusion, leave no orphan `tool_use` blocks

```
/**
 * Step 1: Consume remaining results from started tools
 *
 * Background: In streaming output, once a tool_use block is fully output, the tool
 * begins executing immediately.
 * When the user interrupts, some tools may:
 *   - Still be in queue (not started)
 *   - Currently executing (in-flight)
 *
 * getRemainingResults() iterates through all submitted tools:
 *   - Not started (in queue): skip directly (don't execute)
```

```
   *   - Currently executing: wait for completion, then generate a synthetic tool_result:
   *       "<tool_use_error>Error: Interrupted by user</tool_use_error>"
   *
   * Key: If getRemainingResults() is not called, these tool_use blocks will never have
   * corresponding tool_result blocks, resulting in an incomplete message tree structure,
   * confusing the API in the next round.
   */
  if (toolUseContext.abortController.signal.aborted) {
    if (streamingToolExecutor) {
      // Actively pull remaining results (drives executor to complete in-flight tools and
generate synthetic errors)
      for await (const update of streamingToolExecutor.getRemainingResults()) {
        if (update.message) yield update.message
      }
    } else {
      // Non-streaming tool execution mode (no streamingToolExecutor):
      // Find all tool_use blocks, inject "Interrupted by user" tool_results one by one
      yield* yieldMissingToolResultBlocks(
        assistantMessages,
        'Interrupted by user',
      )
    }

    /**
     * Step 2: Computer Use Cleanup (CHICAGO_MCP main thread only)
     *
     * In Computer Use mode:
     *   - The main thread holds a process-wide lock (isLockHeldLocally)
     *   - The main thread holds a hidden window (isHidden)
     *
     * When user interrupts:
     *   - Release the lock (let other tasks continue)
     *   - Unhide the window (restore window visibility)
     *
     * Main thread only: subagent cleanup would incorrectly release the main thread's lock,
     * causing the main thread to subsequently see isLockHeldLocally()===false and stop
sending exit notifications.
     */
    if (feature('CHICAGO_MCP') && !toolUseContext.agentId) {
      try {
        const { cleanupComputerUseAfterTurn } = await import(
          './utils/computerUse/cleanup.js'
        )
        await cleanupComputerUseAfterTurn(toolUseContext)
      } catch {
        // Fail silently: this is dogfooding cleanup, not on the critical path
      }
    }

    /**
     * Step 3: Send user interruption message
     *
     * Note: The submit-interrupt scenario (user submits a new message while interrupting
```

```
the current turn)
     * does NOT need this message — because the user's next submitted message already
provides sufficient context,
     * sending an extra "Interrupted by user" message would create information noise.
     * Detection: abort reason === 'interrupt' (triggered by submitInterrupt)
     */
    if (toolUseContext.abortController.signal.reason !== 'interrupt') {
      yield createUserInterruptionMessage({
        toolUse: false,  // Interrupted model thinking/output, not a tool-bearing turn
      })
    }

    return { reason: 'aborted_streaming' }
  }
```

8.2 Tool Execution Interruption

```
if (toolUseContext.abortController.signal.aborted) {
  // ... cleanup logic same as 8.1
  if (toolUseContext.abortController.signal.reason !== 'interrupt') {
    yield createUserInterruptionMessage({ toolUse: true })
  }
  if (maxTurns && nextTurnCountOnAbort > maxTurns) {
    yield createAttachmentMessage({ type: 'max_turns_reached', ... })
  }
  return { reason: 'aborted_tools' }
}
```

8.3 Submit Interrupt

Skip sending the interruption message (`reason !== 'interrupt'`), because the subsequent queued user message already provides sufficient context.

IX. Async Prefetch & Parallelization

9.1 Memory Prefetch

```
using pendingMemoryPrefetch = startRelevantMemoryPrefetch(state.messages, toolUseContext)
// `using` declaration ensures automatic resource cleanup on all exit paths

// In the API streaming loop: consume prefetch results (only when settled)
if (pendingMemoryPrefetch.settledAt !== null && pendingMemoryPrefetch.consumedOnIteration
=== -1) {
  const memoryAttachments = filterDuplicateMemoryAttachments(
    await pendingMemoryPrefetch.promise,
    toolUseContext.readFileState,
  )
  for (const memAttachment of memoryAttachments) {
    yield createAttachmentMessage(memAttachment)
    toolResults.push(msg)
  }
```

```
  pendingMemoryPrefetch.consumedOnIteration = turnCount - 1
}
```

- Uses `using` (TC39 Resource Management) to ensure `pendingMemoryPrefetch` is cleaned up on all exit paths
  - Regardless of how the query loop exits, background memory search is cancelled and logs are recorded
- `consumedOnIteration` prevents duplicate consumption within the same iteration cycle (yield to user) — note this field is compared **between iterations** (`turnCount - 1`), not within the same iteration

```
什么时候 prefetch 会被"消费"（yield 给用户）？

1. prefetch 的 Promise 已经有结果了（settledAt !== null）
2. 之前没有被消费过（consumedOnIteration === -1）

消费之后，立刻把当前轮次记录下来（consumedOnIteration = turnCount - 1）。

为什么要用 turnCount - 1？

因为 prefetch 只能在工具执行完毕之后才会被检查和消费。来看一轮 query 循环的内部顺序：

第 N 轮
├── 预检阶段（SNIP / Microcompact / Autocompact）
├── API 流式循环 ← prefetch 在这里 settle（Promise 有结果了）
├── 工具执行（StreamingToolExecutor / runTools）← 这里是唯一检查 prefetch 的地方
└── → continue → 进入第 N+1 轮

prefetch 在 API 流式循环中就 settle 了，但代码检查 prefetch
的位置在"工具执行之后"——工具执行本身发生在同一轮里，所以 prefetch 实际上是在第 N
轮的工具执行完毕处被消费的，不是等到第 N+1 轮。
```

- `readFileState` filters out file states the model has already processed — this is cross-iteration cumulative filtering, ensuring that even if prefetch results are consumed multiple iterations later, file states the model already "knows" are not injected again.

9.2 Skill Discovery Prefetch

```
const pendingSkillPrefetch = skillPrefetch?.startSkillDiscoveryPrefetch(
  null, messages, toolUseContext,
)
// Consumed asynchronously during tool execution
```

PS:

- Note this section has no concrete code implementation; it is inferred from interfaces and comments. The following important technical details are unknown:
  - How startSkillDiscoveryPrefetch is implemented internally (what model, what prompt)
  - The specific judgment logic of findWritePivot
  - How collectSkillDiscoveryPrefetch calculates the hidden_by_main_turn boolean
  - What source of skills it searches (filesystem? vector retrieval?)
- Skill Discovery Prefetch is a background async discovery + consumption-with-tool-results mechanism: in tool turns that may involve write operations, it uses findWritePivot to determine when to trigger background Skill Discovery, quietly prepares possibly needed skill attachments while the model reasons and tools execute, and finally injects them along with tool results, using hidden_by_main_turn to ensure the user never perceives the discovery latency — transforming the per-turn blocking cost into an almost free side effect.

9.3 Tool Use Summary

```
nextPendingToolUseSummary = generateToolUseSummary({
  tools: toolInfoForSummary,
  signal: toolUseContext.abortController.signal,
  isNonInteractiveSession: toolUseContext.options.isNonInteractiveSession,
```

```
  lastAssistantText,
})
.then(summary => summary ? createToolUseSummaryMessage(summary, toolUseIds) : null)
.catch(() => null)
```

After each turn's tool execution completes, package "which tools were executed, what did they do" and send it to Haiku (small model), letting Haiku generate a one-line human-readable summary (e.g., "Fixed NPE in AuthService"), displayed in the mobile UI:

- The result is yielded at the next turn's `pendingToolUseSummary` position
- Sub-agents (`toolUseContext.agentId`) skip this step (not displayed in mobile UI)

PS: The model cannot see this summary (context is unchanged); it is user-exclusive information (displayed in the UI).

X. Tool Use Context Refresh

```
if (updatedToolUseContext.options.refreshTools) {
  const refreshedTools = updatedToolUseContext.options.refreshTools()
  if (refreshedTools !== updatedToolUseContext.options.tools) {
    updatedToolUseContext = {
      ...updatedToolUseContext,
      options: { ...updatedToolUseContext.options, tools: refreshedTools },
    }
  }
}
```

- Called whenever tools are needed, always returns the latest state
- Ensures newly connected MCP servers become available promptly across turns.

Why `refreshTools` is a function rather than a value

`toolUseContext.options.refreshTools` is a **function** (`() => Tools[]`), not a snapshot of the tool array. The motivation behind this design choice: **the tool list may change during turn execution**.

Typical scenario: A user connects a new MCP server during tool execution. MCP server connections are async events that can occur at any time. If `refreshTools` were a snapshot value rather than a function, the query loop would need to re-call it on every possible exit path to ensure it sees the latest tool list — easy to miss and inelegant.

By designing it as a function, the query loop only needs to **call on demand** (`refreshTools()`) when it needs the tool list, with each call returning the current latest state — this is essentially **lazy evaluation** applied to tool management: don't lock in the tool list at the start of a turn, but fetch the latest value each time it's needed.

XI. Error Classification & Handling

11.1 Error Type Mapping

Error	Handling	Exit Reason
`ImageSizeError` / `ImageResizeError`	Yield user-friendly message	`image_error`

Error	Handling	Exit Reason
Model runtime exception	Yield orphan tool_result + real error message	`model_error`
`FallbackTriggeredError`	Switch model, retry	Continue loop
`PROMPT_TOO_LONG` (pre-call)	Yield error message	`blocking_limit`
413 withheld → recovery failed	Expose withheld error	`prompt_too_long`
Media error recovery failed	Expose withheld error	`image_error`
Stop hook blocking	Inject error message, continue	Continue loop
Tool execution interruption	Generate synthetic tool_result	`aborted_tools`
`hook_stopped_continuation`	Stop	`hook_stopped`
max_turns reached	Yield attachment message	`max_turns`
Stop hook prevented	Stop	`stop_hook_prevented`

11.2 Safety Guarantees for Error Exposure

```
// Prevent exposing misleading "[Request interrupted by user]"
// SDK consumers might see a phantom interrupt and ignore the real cause
yield createAssistantAPIErrorMessage({ content: errorMessage })
logAntError('Query error', error)  // Loud logging in ants mode
return { reason: 'model_error', error }
```

PS:

- This is an error attribution transparency guarantee — exposing a **"looks like user cancellation, actually an SDK bug"** phantom interrupt as the real error prevents misdiagnosis, helping real bugs be discovered and fixed.
- The actual situation may be a Node.js runtime exception (e.g., a method that doesn't exist), but the outermost SDK layer doesn't handle it finely, wrapping it uniformly as "Request interrupted by user" -> queryModelWithStreaming can directly throw exceptions in very rare cases (rather than normally yielding errors).

XII. Feature Flag Summary

Feature Name	Purpose	Related File
`REACTIVE_COMPACT`	Enable reactive compression	`services/compact/reactiveCompact.js`
`CONTEXT_COLLAPSE`	Enable context collapse	`services/contextCollapse/index.js`
`HISTORY_SNIP`	Enable historical message pruning	`services/compact/snipCompact.js`
`CACHED_MICROCOMPACT`	Enable cached microcompression	`compact.ts`
`TOKEN_BUDGET`	Enable token budget early stopping	`query/tokenBudget.ts`
`UNATTENDED_RETRY`	Enable persistent retry	`services/api/withRetry.ts`
`BG_SESSIONS`	Enable background task summarization	`utils/taskSummary.js`
`CHICAGO_MCP`	Enable MCP cleanup hooks	`utils/computerUse/cleanup.js`
`EXTRACT_MEMORIES`	Enable memory extraction	`services/extractMemories/`
`EXPERIMENTAL_SKILL_SEARCH`	Enable skill pre-search	`services/skillSearch/prefetch.js`

Feature Name	Purpose	Related File
`TEMPLATES`	Enable template task classification	`jobs/classifier.js`

All features are accessed via `feature('NAME')`, with `bun:bundle`'s tree-shaking ensuring that disabled feature strings don't enter external builds.

12.1 `querySource` as a Typed Safety Policy Context

`querySource` is an **imperative label** answering the question "who is calling query()", not just a string identifier. It is used as the basis for policy dispatch:

- `"compact"` / `"session_memory"`: Sub-agent, skip blocking limit check (deadlock protection)
- `"repl_main_thread"` / `"sdk"`: Main session, save CacheSafeParams
- `"agent:*"`: Sub-agent, persist tool result replacement records
- Background sources: Fail immediately on 529, don't retry

The key insight of this design: **the caller knows its own semantics better than the query loop itself**. The query loop doesn't need to understand "what a compact agent is", only that "compact-source calls need to be exempt from blocking checks" — this policy knowledge is passed from the caller to query, not derived by query itself.

XIII. Design Philosophy Summary

> Before reading the following sections, please review the **"Three Design Principles"** from Section I: state machine pattern, unified error recovery exit, and cooperative concurrency. These principles permeate throughout query.ts and are the foundation for understanding the motivation behind every specific mechanism.

13.0 The "Three-Layer Funnel" Philosophy of Error Handling

query.ts follows a unified "three-layer funnel" philosophy for error handling, rather than writing separate branches for each error type:

- **Layer 1 (Withhold)**: Capture recoverable errors in the streaming loop, but don't expose them to the caller. Reason: SDK consumers may terminate the session when they see the `error` field, preventing subsequent recovery logic from ever executing.
- **Layer 2 (Recover)**: Attempt automatic recovery after a turn ends: PTL goes collapse drain → reactive compact → eventually expose; max_output_tokens goes 8k→64k escalation → multi-turn recovery message → eventually expose; 529 goes exponential backoff → model fallback → eventually expose.
- **Layer 3 (Expose)**: After all recovery paths are exhausted, expose the error and exit. At this point, `executeStopFailureHooks` is called (not the full stop hooks chain), preventing the death spiral caused by stop hooks injecting more tokens when the model never produced a valid response.

13.1 Defensive Streaming Processing

Unlike the traditional "call API fully first, then process results" pattern, query.ts processes tools, errors, and interruptions **in real-time** during streaming output. The advantages of this design:

- **Parallelization**: Tools begin executing while the model is still producing output
- **Low latency**: Errors are captured immediately upon occurrence, rather than waiting for a complete response
- **Resource safety**: Long-running tasks can be interrupted immediately

13.2 Withheld Mode

For recoverable errors (413 PTL, max_output_tokens, media size), choosing to **withhold** rather than expose immediately is because SDK consumers terminate the session upon seeing an `error` field, preventing recovery logic from ever executing. The withheld mechanism ensures recovery paths have sufficient opportunity to complete.

13.3 Immutable State Transitions

All loop state updates are done by **replacing the entire State object**:

```
state = { ...state, newField: newValue }
```

Instead of modifying fields one by one, which makes:

- State structure consistent across all continue jump sites
- Easy to test recovery paths via the `state.transition` field
- Avoids bugs where "some fields are not correctly reset in some branches"

13.4 Multi-Layer Degradation Strategy

Each error type is designed with **at least two layers** of degradation paths:

- Prompt Too Long: `pre-call blocking` → `context collapse drain` → `reactive compact` → `expose error`
- Max Output Tokens: `8k→64k escalation` → `multi-turn recovery message` → `expose error`
- 529 Overload: `retry×3` → `model fallback` → `expose error`

13.5 Dependency Injection: Intentional Design for Testability

query.ts's `deps.ts` implements a Dependency Injection (DI) pattern, specifically addressing the testing problem: at the time, 6-8 test files each used `spyOn` to intercept functions from different modules, causing import + spy boilerplate to proliferate.

```
// Design principles of deps.ts:
// 1. Types derived from real implementations (typeof fn), ensuring signatures stay in sync
// 2. Production uses productionDeps() to get real implementations
// 3. Tests inject fake implementations via QueryParams.deps
// 4. Scope deliberately narrowed (only 4 dependencies), expand only after proving the
pattern works
```

This is "minimal viable DI": instead of introducing an IOC container across the entire system, it provides a thin injection layer in the module that most needs testability (query.ts). The `deps` object itself is a plain TypeScript object (not a class/container), requiring no framework support for injection.

13.6 Protection Against 529 Overload "Amplification Attacks"

The global design consideration implicit in the 529 error handling: when the API Gateway is in a capacity overload (529) state, **each retry request actually increases the burden on the gateway**, forming a positive feedback loop — this differs from ordinary 429 rate limiting, where local quota exhaustion means retries don't affect others.

The `FOREGROUND_529_RETRY_SOURCES` design in the code is the direct response to this problem: only query sources where the user is waiting for results in the foreground will retry on 529; all background tasks

(summarization generation, title generation, classifiers, etc.) **fail immediately** on 529, never retrying. This minimizes the request amplification factor during 529 — because background task failures don't affect user experience, while foreground user requests can briefly wait and retry.

Also note: when 529 triggers model fallback, only **non-subscribing users** (`!isClaudeAISubscriber()`) or non-custom Opus models fall back. Paying users don't follow the fallback path — their enterprise agreements typically include PAYG (pay-as-you-go), making retries more reasonable than fallback.

13.7 Avalanche Isolation for Background Tasks

Another design point related to 529 overload protection: background tasks (`session_memory` queries, `compact` agents, etc.) encountering context overflow trigger a deadlock — because these subprocesses themselves need to run to **reduce** token usage, and if they're blocked at invocation time due to too many tokens, they'll never be able to execute.

The code reflects this isolation principle in multiple places:

- Blocking limit checks skip `compact` and `session_memory` query sources
- Microcompact skips `compact` and `session_memory` query sources (preventing subagent and parent agent from both attempting compression and waiting for each other)

This is essentially a **resource-order deadlock prevention**: if A needs B to complete first to run, and B also needs A to complete first to run, the system deadlocks. By exempting "resource-consuming subtasks" from being blocked by resource limits, this classic deadlock pattern is avoided.

13.8 QueryCheckpoint's Dual-Layer Performance Tracking System

The `queryCheckpoint()` calls scattered throughout query.ts actually belong to a **dual-layer performance tracking system** serving different use cases:

Layer 1: Headless Profiler (`headlessProfilerCheckpoint`) tracks **end-to-end latency breakdown from the user pressing Enter to seeing the first character** in headless mode. This data is sent to a centralized monitoring system for on-call engineers to determine whether scaling or rollback is needed.

Layer 2: Query Profiler (`queryCheckpoint` + `utils/queryProfiler.ts`) tracks **per-stage timing within a single query**, locating "where this query was slow". This data is enabled via the `CLAUDE_CODE_PROFILE_QUERY=1` environment variable and output to debug logs during local development.

The layered design of both avoids "using one tool to satisfy both monitoring and debugging needs, doing neither well." Monitoring requires low overhead, good aggregation, and no interference with production services; debugging requires granularity, immediate feedback, and local analysis.

13.9 "Implicit Dependency" Awareness in Tool Execution

The design of StreamingToolExecutor's cascade cancellation triggered by Bash errors (sibling cancellation) reveals a profound understanding of **implicit dependencies**:

- Multiple Bash commands typically have implicit file/directory dependencies between them. The cascade cancellation mechanism of the `hasErrored` flag is precisely an encoding of this implicit dependency: by letting the first Bash error "kill" all currently executing sibling tools, the system acknowledges the dependency relationship between Bash commands and stops the bleeding immediately.
- But this doesn't mean all tools follow the same rule — read-only tools (Read/WebFetch/Grep/Glob) are **concurrent-safe**; there are no implicit dependencies between them, so they can run in parallel. This is a

refined concurrency control strategy: conservative (serial) for operations with uncertain dependencies, aggressive (parallel) for operations with certain no dependencies.

13.10 Analytics as a First-Class Citizen of Architecture

Throughout query.ts, there are numerous `logEvent()` calls scattered — not "post-hoc instrumentation added late in development", but an integral part of the system architecture.

Analytics events are designed alongside the feature development itself, not retrofitted after launch. Every key business decision point (autocompact trigger, model fallback, token budget stop, streaming tool execution) simultaneously outputs structured data fields. These fields are designed from the start to answer specific product questions (e.g., "how much did compression actually save in costs", "was token budget stopped proactively or passively"), allowing the product team to make decisions based on data rather than guesswork or log inspection. Product logic and analytics logic are the same code, the same PR, the same review — field naming is consistent, data format is consistent, validated before launch, and historical data is traceable. This transforms "data-driven decision making" from talk into a hard-coded institution in the code.

13.11 `using` Declaration & Deterministic Resource Management

Memory Prefetch uses the TC39 `using` declaration (resource management) rather than manual cleanup — this is a guarantee of **deterministic resource release**:

```
using pendingMemoryPrefetch = startRelevantMemoryPrefetch(state.messages, toolUseContext)
```

`using` is JavaScript's deterministic destruction mechanism (similar to Rust's `Drop` or Python's `with`). Whether the code exits the query loop via `return`, `throw`, or `continue`, the `pendingMemoryPrefetch` cleanup logic will be called. This eliminates an entire class of "resource leak" bugs — developers don't need to remember to manually call cleanup on every exit path.

In a complex loop like query.ts with 9 different exit paths, deterministic cleanup is especially important. Any single exit path missing cleanup accumulates leaks over long sessions.

13.12 Death Spiral Protection

Multiple locations in the code explicitly avoid falling through to stop hooks when **the model has not produced a valid response**:

```
// Do NOT fall through to stop hooks: the model never produced a valid response,
// so hooks have nothing meaningful to evaluate. Running stop hooks
// on prompt-too-long creates a death spiral:
// error → hook blocking → retry → error → …
yield lastMessage
void executeStopFailureHooks(lastMessage, toolUseContext)
return { reason: isWithheldMedia ? 'image_error' : 'prompt_too_long' }
```

Similarly, the `hasAttemptedReactiveCompact` persistence design prevents stop hook blocking errors from triggering compact into an infinite loop.

XIV. Key Data Flow Diagram

```
User Input
  |
```

```
  ▼
┌──────────────────────────────────────────────────────────────┐
│  queryLoop: while(true)                                      │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Pre-check Phase:                                      │  │
│  │  1. Memory Prefetch (async, non-blocking)            │  │
│  │  2. Skill Discovery Prefetch (async)               │  │
│  │  3. HISTORY_SNIP ──→ Free up token budget          │  │
│  │  4. Microcompact ──→ Cache optimization            │  │
│  │  5. Context Collapse ──→ Collapse                  │  │
│  │  6. Autocompact ──→ Summarization compression      │  │
│  │  7. [pre-call] Blocking limit check                │  │
│  └──────────────────────────────────────────────────────┘    │
│                          │
│                          ▼
│  ┌──────────────────────────────────────────────────────┐    │
│  │ API Call Phase (withRetry wrapper)                 │  │
│  │  for await (message of deps.callModel())           │  │
│  │   ├─ Streaming tool registration → StreamingToolExecutor │
│  │   ├─ Real-time tool execution → getCompletedResults() │
│  │   ├─ Withheld checks (PTL/Media/MOT)               │  │
│  │   └─ Orphan message Tombstoning (on fallback)    │  │
│  │                                                  │  │
│  │  Error handling:                                   │  │
│  │   ├─ 529 → Exponential backoff → FallbackTriggeredError │
│  │   ├─ 429/401/408/409/5xx → Exponential backoff retry  │
│  │   └─ max_tokens overflow → Dynamically lower max_tokens │
│  └──────────────────────────────────────────────────────┘    │
│                          │
│                          ▼
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Post-processing Phase:                             │  │
│  │   ├─ User interruption check → Synthetic tool_result │
│  │   ├─ Post-sampling Hooks (fire-and-forget)       │  │
│  │   ├─ needsFollowUp=false?                          │  │
│  │   │   ├─ PTL recovery: collapse drain → reactive compact │
│  │   │   ├─ MaxOutputTokens: escalate → multi-turn recover → expose │
│  │   │   ├─ Stop Hooks → blockingErrors → continue  │  │
│  │   │   ├─ Token Budget → continue/stop              │  │
│  │   │   └─ return { reason: 'completed' }          │  │
│  │   └─ needsFollowUp=true?                           │  │
│  │       ├─ Tool execution (Streaming / Blocking)    │  │
│  │       ├─ Memory Prefetch consumption             │  │
│  │       ├─ Skill Discovery consumption             │  │
│  │       ├─ MCP tool refresh                          │  │
│  │       └─ continue (next turn)                    │  │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

XV. Quick Reference: Related Files

File	Responsibility
`restored-src/src/query.ts`	Main loop, context management, streaming fault tolerance, recovery path orchestration
`restored-src/src/query/stopHooks.ts`	Stop/TaskCompleted/TeammateIdle hooks execution
`restored-src/src/query/tokenBudget.ts`	Token budget tracking & early-stop decision
`restored-src/src/query/deps.ts`	Dependency injection (`deps.microcompact` / `deps.autocompact` / `deps.callModel`)
`restored-src/src/query/config.ts`	Immutable config snapshot (except Statsig gates)
`restored-src/src/services/api/withRetry.ts`	API retry wrapper, backoff strategy, model fallback
`restored-src/src/services/compact/reactiveCompact.ts`	Reactive compression (413/media error recovery)
`restored-src/src/services/compact/autoCompact.ts`	Proactive compression (per-turn threshold trigger)
`restored-src/src/services/compact/compact.ts`	`buildPostCompactMessages` compression result construction
`restored-src/src/services/compact/snipCompact.ts`	Historical message pruning
`restored-src/src/services/compact/microcompact.ts`	Microcompression / Cached Microcompact
`restored-src/src/services/contextCollapse/index.ts`	Context collapse
`restored-src/src/services/tools/StreamingToolExecutor.ts`	Streaming tool parallel executor
`restored-src/src/services/tools/toolOrchestration.ts`	`runTools` blocking tool execution
`restored-src/src/utils/hooks.js` / `postSamplingHooks.ts`	Hook system
`restored-src/src/utils/tokenBudget.js`	`getBudgetContinuationMessage` progress message
`restored-src/src/bootstrap/state.ts`	Global token budget state